

The Complete Git & GitHub Reference Handbook

A Practical Guide & Architecture Manual for Software Developers

REFERENCE MANUAL • JUNIOR & INTERN EDITION

O. Git Concepts Explained Simply

Before executing commands, it is crucial to understand Git's core architecture. Git does not simply track individual file deltas; it snapshots information across three localized conceptual states and coordinates transitions into cloud-hosted platforms.

The Three Local Trees + Remote Architecture

1. **Working Directory:** The local scratchpad containing physical files currently being updated or authored on your machine's system workspace.
2. **Staging Area (Index):** A technical preparation floor. This layer behaves as a pre-commit review station holding precise delta contents aggregated to build the next valid historical frame.
3. **Local Repository (.git):** The localized historical database container enclosing all packaged commits, snapshot objects, structural trees, and references.
4. **Remote Repository:** The synchronized network version of your tracking records hosted on systems like GitHub or GitLab used to share code.

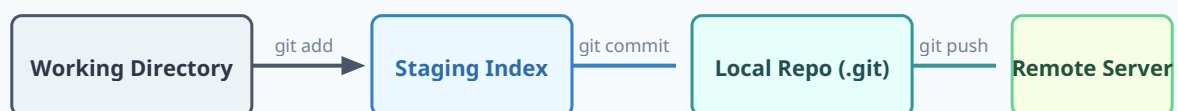


Figure 1.1: Local Workspace Trees to Remote Data Pipeline Layout

Core Terminology Definitive Matrix

- **Commit:** A cryptographic, atomic structural snapshot of staged components mapped at a point in linear history, tagged securely by a unique SHA-1 checksum identity token.

- **Branch:** A lightweight, highly dynamic mutable pointer targeting the leaf tip node of a collection of chronological commit sequences.
- **HEAD:** An active reference variable indicating the context location of your current running local workspace branch ecosystem.
- **Detached HEAD:** An architectural anomaly arising when checking out an absolute commit entry sequence directly rather than a branch label tracking context, meaning newly built frames stand unprotected against garbage cycles.
- **Origin & Upstream:** `origin` specifies the foundational remote target where your personalized forked repository exists online. `upstream` maps the foundational root core repository owned by a collective team ecosystem from which synchronization actions take place.

A. Initial Setup (One-Time Setup)

1. System Identity Configurations

Before launching local commit commands, you must configure global meta-tags to correctly attribute project additions:

```
# Establish global attribution fields
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Set the default baseline master tracking branch structural tag
git config --global init.defaultBranch main
```

2. Active Settings Auditing

```
git config --list
```

This command lists all combined global, system, and local configurations acting on the file framework directory ecosystem.

3. Cryptographic Key Integration (SSH Verification)

Using SSH keys bypasses repetitive manual user-token checks and safeguards connection tunnels securely.

```
# Step 1: Create local keys via Ed25519 scheme
ssh-keygen -t ed25519 -C "your.email@example.com"

# Step 2: Initialize background secure agent environment
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Print and copy the generated public output payload (`cat ~/.ssh/id_ed25519.pub`) into GitHub's portal interface under **Settings -> SSH Keys**. Validate the channel manually using:

```
ssh -T git@github.com
```

Common Mistake: Profile Identity Mismatch

If your global setting `user.email` differs from the verified contact email strings saved in your personal GitHub accounts, commits show up completely unlinked without user profile tracking or contribution chart inclusion. Rectify errors via:

```
git config --global --replace-all user.email "correct@email.com" .
```

B. Starting a New Project

Architectural Paradigm: Initialization vs. Cloning

Vector Checkpoint	git init	git clone
Context Target	Turns a local unversioned directory into a Git repository.	Copies an existing remote repository locally.
Remote Bindings	Zero tracking nodes default; must add manual targets.	Automatically configures <code>origin</code> tracking links.
Ancestral History	Generates a completely blank timeline starting at zero.	Downloads full branch snapshots and tree frames.

1. Instantiating via Local Baseline Shell

```
mkdir corporate-dashboard && cd corporate-dashboard
git init
```

2. Mirroring Remote Workspaces

```
git clone git@github.com:organization/corporate-dashboard.git
```

3. Binding Local Project Workspaces to Remote Clouds

If initialized locally, connect your workspace to a blank remote repository template:

```
# Map the remote cloud route to target variable "origin"
git remote add origin git@github.com:your-username/corporate-dashboard.git

# Stream local states up to establish upstream link patterns
git push -u origin main
```

C. Daily Development Workflow

Command Execution Matrix

1. git status

Displays modifications in the working directory and tracks files added to the staging area.

```
git status
git status -s # Returns clean, abbreviated tabular notation blocks
```

2. git add

Stages changed file items to prepare them for the next history commit snapshot.

```
git add src/components/Login.js # Targeted single-file staging
git add . # Structural staging of all current directory items
git add -p # High-fidelity interactive patch block routing selecti
```

3. git commit

Saves staged modifications into a permanent local history commit frame block.

```
git commit -m "feat: complete user auth layer middleware integration"
git commit -am "chore: streamline styles configuration across modules" # Bypass staging
git commit --amend # Modify or correct the very last local commit tracking record
```

4. Synchronization Operations

- `git fetch` : Downloads latest metadata commits, branch indices, and structural object logs from remote clouds without overwriting local file systems.
- `git pull` : Instantly pulls remote commits and automatically merges them into your current active local branch.

D. Branch Management

Isolating features inside decoupled branches ensures that production-ready main lifelines remain completely safe and unbroken during active engineering sprints.

1. Branch Navigation & Operations

```
# Instantiation and switching via clean modern conventions
git switch -c feature-payment-gateway

# Review absolute state metrics on all branches
git branch -a # Prints local and remote tracking routes side by side

# Modify labels cleanly
git branch -m feature-stripe-api

# Housekeeping purges
git branch -d feature-stripe-api # Safe block option; errors out if unmerged
git branch -D feature-stripe-api # Destructive force override option
```

2. Merging Integrations

Combines code histories from target tracking structures back into your active branch framework.

```
git switch main
git merge feature-stripe-api
```

Fast-Forward vs. Three-Way Merge Execution

If the target `main` branch has received no intermediate updates since your feature split, Git performs a fast-forward merge by shifting the tracking reference pointer directly to the tip of your feature branch. If `main` has diverged, Git creates a dedicated **Three-Way Merge Commit** that acts as a structural bridge joining both separate histories.

E. Working with Remote Repositories (The Fork Workflow)

In collaborative enterprise setups, direct commit permissions on main operational pipelines are heavily restricted. Teams manage additions using the **Fork Workflow Layout**.

Synchronizing an Upstream Open Source Project

```
# Step 1: Bind your local project to the parent upstream code vault
git remote add upstream git@github.com:core-organization/main-framework.git

# Step 2: Query for paths confirmation
git remote -v

# Step 3: Fetch structural update tracks from parent vault nodes
git fetch upstream

# Step 4: Safely combine baseline framework changes into local tracking paths
git switch main
git merge upstream/main
```

F. Git Log & History Inspection

Comprehensive Log Traversal Guide

Operational Command Syntax	Target Intent & Audit Context	Visual Presentation Pattern
<code>git log --online</code>	High-level review of project timelines.	Abbreviated single-line records showing SHA and title fields.
<code>git log --graph --online --all</code>	Visualizing project branches and merges.	ASCII tree lines illustrating branch paths.
<code>git log --author="Alex"</code>	Auditing specific individual author outputs.	Filters and lists only commits matching the specified name.
<code>git log --since="10 days ago"</code>	Tracking code metrics during active sprints.	Time-bounded tracking outputs.
<code>git log -p</code>	Code review and patch evaluations.	Deep line-by-line file difference patch files.
<code>git log -S "PAYMENT_KEY"</code>	Tracking sensitive code updates or string additions.	Filters logs by occurrences of specific code strings.

High-Fidelity Code Auditing

```
# Track author changes line-by-line across a specific file
git blame src/utils/crypto.js

# Review the entire commit history for a single file
git log -- src/utils/crypto.js
```

G. Understanding Git Differences

The `git diff` command performs structural matrix evaluations across different states of your project files.

```
git diff          # Compares modifications in the Working Directory against Staging Index
git diff --staged # Compares modifications in Staging Index against Local Git History database
git diff main..feature-auth # Compares code deltas between two target branch heads
```

H. Stashing Work

Stashing allows you to save your current uncommitted changes to a temporary workspace stack so you can switch branches or apply critical hotfixes without committing incomplete work.

```
git stash save "WIP: redesigning table column templates" -u # Stash work including untracked files
git stash list      # View the saved states index stack
git stash pop       # Reapply the last stashed state and remove it from the stack
git stash apply stash@{0} # Reapply a specific stashed state while keeping it in the stack
```

I. Undoing Mistakes

Command Operation	Target Layer	Safety Category	Workspace Impact Status
<code>git restore <file></code>	Working Directory	DANGEROUS	Permanently discards local uncommitted modifications.
<code>git reset --soft HEAD~1</code>	History Database	SAFE	Rolls back commits while preserving all modifications in Staging.
<code>git reset --mixed HEAD~1</code>	History Database	SAFE	Rolls back commits and unstages changes, keeping them in Working Directory.
<code>git reset --hard <hash></code>	All Layers	CRITICAL	Destructive wipe. Deletes all local changes and commits permanently.
<code>git revert <hash></code>	History Database	SAFE	Appends a new commit that safely rolls back changes without rewriting history.

J. Merge Conflicts

Merge conflicts occur when separate tracking streams apply conflicting modifications to the exact same line of code inside a file, requiring manual developer intervention to resolve.

Conflict Marker Topography

```
<<<<<<< HEAD
const apiUrl = "https://api.internal.prod/v1";
=====
const apiUrl = "https://api.external-vendor.com/v2";
>>>>>>> feature-vendor-migration
```

- **HEAD** represents your current local branch configuration state.
- **=====** serves as the structural division center boundary.
- **feature-vendor-migration** indicates the incoming code changes.

Resolution Workflow Protocol

1. Open the conflicting file inside your code editor and select the correct line configuration.
2. Completely delete the structural conflict markers (**<<<<<<<** , **=====** , **>>>>>>>**).

3. Stage the conflict-resolved file tracking path: `git add src/config.js`.
4. Finalize the integration commit frame cleanly: `git commit -m "fix: resolve merge conflicts in API configuration"`.

K. Synchronizing with Team Members

Run precise queries to monitor tracking differences between your local branch and remote repositories:

```
# Fetch remote update data parameters
git fetch origin

# Count the commits your local branch is behind the remote tracking line
git rev-list --count HEAD..origin/main

# Count the commits your local branch is ahead of the remote tracking line
git rev-list --count origin/main..HEAD
```

L. Advanced Investigation Commands

- `git reflog` : Displays a complete structural log tracking pointer movements across all branches. This command serves as a critical recovery tool for locating and restoring deleted branches or lost commits.
- `git show <hash>` : Displays full author details and line-by-line code updates for a specific commit hash.

M. Common Real-World Scenarios

1. Fetching Latest Sprint Code Updates

```
git switch main
git pull origin main
```

2. Pushing a Local Feature Branch for the First Time

```
git switch -c feature-analytics-tracker
git add .
git commit -m "feat: integrate tracker hooks"
git push -u origin feature-analytics-tracker
```

3. Fixing a Commit Made to the Wrong Branch

```
# Step 1: Create a feature branch from your current position to safeguard your work
git branch feature-saved-work

# Step 2: Roll back your current branch by one commit to remove the changes
git reset --hard HEAD~1

# Step 3: Switch to your newly created branch to continue working
git switch feature-saved-work
```

N. Git Command Cheat Sheet

Developer Objective Situation	Tactical Git Command Formula
Check status of working directory	<code>git status</code>
Stage all modified and untracked files	<code>git add .</code>
Commit staged changes with a clear message	<code>git commit -m "message"</code>
Push local branch to remote repository for the first time	<code>git push -u origin <branch></code>
Temporarily save dirty workspace changes without committing	<code>git stash</code>
Restore stashed changes and remove them from the stack	<code>git stash pop</code>

Core Engineering Interview Mastery

Q1: What is the mechanical difference between git merge and git rebase operations?

Answer: `git merge` combines separate branches by creating a new merge commit that preserves the historical context of both timelines. `git rebase` updates your branch's base commit by rewriting history and reapplying your commits sequentially on top of the target branch tip. This creates a clean, linear project timeline.

Q2: What occurs structurally during a Detached HEAD state, and how do you resolve it?

Answer: A detached HEAD state occurs when your `HEAD` pointer targets a specific commit hash or tag directly rather than tracking an active branch pointer. To safely preserve changes made in this state, initialize a new branch pointer from your position by running: `git switch -c rescue-branch-name`.